

Question		Marks
14	1	All marks for AO3 (programming) Mark points 1 to 3 relate to the <code>Ant</code> class. - Adding a collection to store positions within the <code>Ant</code> class; A. by index or coordinate. A. Storing unique values or lists with duplicates; <code>AdvanceStage</code> contains selection structure comparing position to <code>Nest</code> location. A. any acceptable method; <code>AdvanceStage</code> clears the list appropriately; Mark points 4 to 10 relate to the <code>WorkerAnt</code> class. - Second selection structure within <code>ChooseCellToMoveTo</code> references visited locations. R. altered pheromone-less behaviour; R. This modifies the 'return to nest' behaviour. - Iterative structure goes through <code>ListOfNeighbours</code>; - Selection statement within the iterative structure to skip/ignore -1; - Valid neighbours are added to a list; - Selection structure for no valid moves correctly exits without the ant moving; - Iterative structure with random position/index generated within; - Iterative structure continues until valid not-visited neighbour is generated. A. random valid neighbour chosen from previously generated list; Mark points 11 relates to the modified <code>Simulation</code> class. - Ant's current location is added to the list after the stage advances. A. Nest location is not added.
14	2	Mark is for AO3 (evaluate) **** SCREEN CAPTURE **** <i>Must match code from 14.1, including prompts on screen capture matching those in code.</i> <i>Code for 14.1 must be sensible.</i> Screen capture(s) showing that after 40 stages, there are still ants. A. Simulations in which all ants are dead, provided code correct code as it is not guaranteed. (~ 70% will have ants.) Expected Solution: Changes to <code>Simulation.AdvanceStage</code> <pre>foreach (Ant A in Ants) { A.AdvanceStage(Nests, Ants, Pheromones); A.visitedPositions.Add(GetIndex(A.GetRow(), A.GetColumn())); ... }</pre>

Modifications to Ant.AdvanceStage

```
public override void AdvanceStage(List<Nest> Nests,
List<Ant> Ants, List<Pheromone> Pheromones)
{
    Stages++;

    if (IsAtOwnNest()) visitedPositions = new
List<int>();
}
```

Modifications to WorkerAnt.ChooseCellToMoveTo

```
public override void ChooseCellToMoveTo(List<int>
ListOfNeighbours, int
IndexOfNeighbourWithStrongestPheromone)
{
    if (AmountOfFoodCarried > 0)
    {
        if (Row > NestRow)
        {
            Row--;
        }
        else if (Row < NestRow)
        {
            Row++;
        }
        if (Column > NestColumn)
        {
            Column--;
        }
        else if (Column < NestColumn)
        {
            Column++;
        }
    }
    else if (IndexOfNeighbourWithStrongestPheromone
== -1
        || visitedPositions.Contains(
IndexOfNeighbourWithStrongestPheromone))
    {
        List<int> validNeighbours = new List<int>();
        foreach (int index in ListOfNeighbours)
        {
            if (index != 0 && !
visitedPositions.Contains(index))
            {
                validNeighbours.Add(index);
            }
        }
        if (validNeighbours.Count() == 0) return;

        int IndexToUse = -1;
```

		<pre>do { IndexToUse = ChooseRandomNeighbour(ListOfNeighbours); } while (visitedPositions.Contains(IndexToUse)); ChangeCell(IndexToUse, ref Row, ref Column); } else { int IndexToUse = ListOfNeighbours.IndexOf(IndexOfNeighbourWithStrongestPheromone); ChangeCell(IndexToUse, ref Row, ref Column); } }</pre>	
--	--	---	--